

Lecture 04: Loading ELF Executable & Procedure Calling Convention

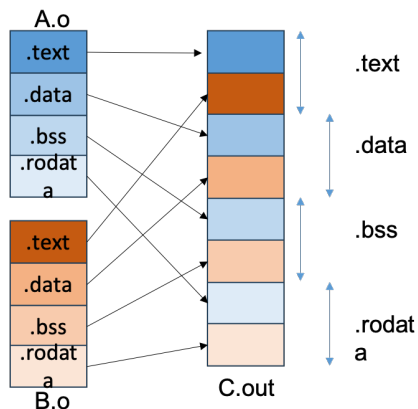
Vivek Kumar

Computer Science and Engineering

IIIT Delhi

vivekk@iiitd.ac.in

Last Lecture



```
[vivek@possum]$ cat fib.c
int get_number();

int fib(int n) {
    if(n<2) return n;
    else return fib(n-1)+fib(n-2);
}

int compute() {
    int n = get_number();
    return fib(n);
}

[vivek@possum]$ cat call-fib.c
int compute();
int get_number() {
    return 40;
}

int main() {
    int result = compute();
    return 0;
}
```

Compilation

fib.o

SYM Name	Type	Location
get_number	External	NULL
fib	Internal	Addr_A
compute	Internal	Addr_B

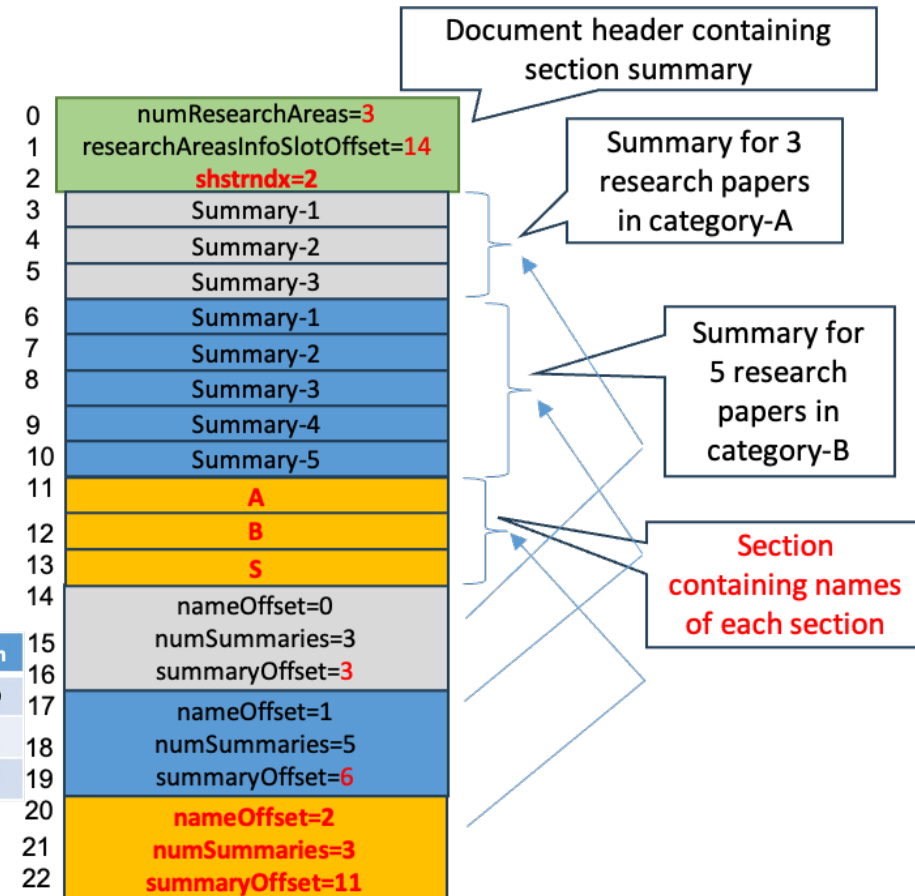
call-fib.o

SYM Name	Type	Location
compute	External	NULL
get_number	Internal	Addr_C

Resolve symbols by linking fib.o and call-fib.o together

a.out

SYM Name	Location
get_number	Addr_D
fib	Addr_E
compute	Addr_F



Class Quiz (1/5)

- Match the following structures with their respective names and responsibilities

(A) Elf32_Phdr	(X) ELF header	(1) Information for Loader
(B) Elf32_Ehdr	(Y) ELF Section Header	(2) Stores roadmap of ELF
(C) Elf32_Shdr	(Z) ELF Program Header	(3) Information for Linker

Class Quiz (2/5)

- What is the meaning of “entrypoint” address in an executable?
 1. Offset of **SHDR** table from the beginning of ELF file
 2. Offset of **PHDR** table from the beginning of ELF file
 3. Address of “main” method
 4. Address of “_start” method

- Where it is stored inside the ELF binaries?
 1. Program header table
 2. Section header table
 3. ELF header
 4. Segments
 5. Sections

Class Quiz (3/5)

- Which of these are required at the compile time and runtime, respectively?
 1. ELF header
 2. PHDR
 3. SHDR
 4. Segments
 5. Sections

Class Quiz (4/5)

- Match the location shown with their respective section name inside the ELF file

```
[vivek@possum:~/elf_os-12$ cat hello.c
#include<stdio.h>
int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";
int main() {
    my_global1 += 1;
    printf("%s = %d\n", str, my_global1);
    return 0;
}
```

Symbol name only in this case

1. .bss
2. .strtab
3. .rodata
4. .text
5. .data
6. .symtab

Class Quiz (5/5)

- Which of these sections appear in “executable” file and which ones in “relocatable” file?
 1. .bss
 2. .dynsym
 3. .shstrtab
 4. .data
 5. .text
 6. .dynstr
 7. .symtab
 8. .strtab

The Story So Far...

- A **cook** (OS) manages a **kitchen** (*computer*)
 - Lecture 01
- The job of a cook is to do **cooking** (*execute binaries*) by reading **recipes** (*executable file*) in a **standard format** (*ELF*)
 - Lecture 02
- The cook has received recipe of a dish in **different files** (*relocatable files*). It will have to use a **stapler** (*linker*) to stitch these files to make a final recipe
 - Lecture 03
- The cook has to **store** (*load*) the **steps** (*instructions*) from the recipe in his memory to start cooking. The cook then execute instructions **one by one** (*call stack*) for cooking
 - **Today's point of discussion!**

Today's Lecture

- Loader
- Call stack
- Procedure calling convention

Loader – High Level Overview

- When an executable file is run, the operating system invokes the loader to load it into memory and start running it
- Loader uses the information stored inside the Program Header Table (PHT)
 - It doesn't require information stored in the Section Header Table (SHT)
 - Linker combines multiple sections of similar type to create one segment
 - There are multiple segments in an executable file, and for each segment, there is one entry in the PHT
 - Several sections can be removed from the executable without affecting program execution (but not all sections can be removed)

Example: Loading Executable in Plain C

```
[vivek@possum]$ cat fib.c
int fib(int n) {
    if(n<2) return n;
    else return fib(n-1)+fib(n-2);
}

int main() {
    int val = fib(40);
    return val;
}
```

```
[vivek@possum]$ gcc -m32 -no-pie -nostdlib fib.c
/usr/bin/ld: warning: cannot find entry symbol _start; de
```

Why ??

Fix: rename
"main" to `_start`

- For simplicity, a) assume there are no APIs from the GNU C library, b) 32-bit compilation, and c) no use of global variables
- The loader has to simply iterate through the PHDR and identify the segment of PT_LOAD type
- Load this segment in memory and jump to [**where?**] for starting the execution

`e_entrypoint`
address in EHDR

Example: Loading Executable in Plain C

```
[vivek@possum]$ readelf -h fib
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:             ELF32
  Data:             2's compleme
  Version:          1 (current)
  OS/ABI:            UNIX - System
  ABI Version:       0
  Type:             EXEC (Executable file)
  Machine:          Intel 80386
  Version:          0x1
  Entry point address: 0x8048141
  Start of program headers: 52 (bytes)
  Start of section headers: 5200 (bytes)
  Flags:            0x0
  Size of this header: 52 (bytes)
  Size of program headers: 32 (bytes)
  Number of program headers: 5
  Size of section headers: 40 (bytes)
  Number of section headers: 15
  Section header string table index: 14
```

```
[vivek@possum]$ readelf -l fib
Elf file type is EXEC (Executable file)
Entry point 0x8048141
There are 5 program headers, starting at offset 52

Program Headers:
  Type   Offset  VirtAddr  PhysAddr  FileSiz MemSiz
  LOAD   0x000000 0x08048000 0x08048000 0x00200 0x00200
  LOAD   0x001000 0x0804a000 0x0804a000 0x0000c 0x0000c
  NOTE   0x0000d4 0x080480d4 0x080480d4 0x00024 0x00024
  GNU_EH_FRAME 0x00016c 0x0804816c 0x0804816c 0x00024 0x00024
  GNU_STACK 0x000000 0x00000000 0x00000000 0x00000 0x00000

Section to Segment mapping:
Segment Sections...
 00  .note.gnu.build-id .text .eh_frame_hdr .eh_frame
 01  .got.plt
 02  .note.gnu.build-id
 03  .eh_frame_hdr
 04
```

```
typedef struct {
    Elf32_Word    p_type;
    Elf32_Off     p_offset;
    Elf32_Addr     p_vaddr;
    Elf32_Addr     p_paddr;
    Elf32_Word     p_filesz;
    Elf32_Word     p_memsz;
    Elf32_Word     p_flags;
    Elf32_Word     p_align;
} Elf32_Phdr;
```

- Array of PHDR with total "e_phnum" number of elements starts at "e_phoff" bytes from the start of the file a.out
- Load the first segment whose "p_type" is "PT_LOAD" in memory. This segment starts at "p_offset"
- Move to the virtual address "e_entrypoint" inside this segment
- Typecast it to the "_start" function pointer type and simply execute!

Loader in OS: In Reality (1/2)

```
vivek@possum:~/os23$ readelf -l hello

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1080
There are 13 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz            MemSiz              Flags     Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x00000000000002d8 0x00000000000002d8 R         0x8
INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
               0x000000000000001c 0x000000000000001c R         0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000660 0x0000000000000660 R         0x1000
LOAD           0x0000000000000100 0x0000000000000100 0x0000000000000100
               0x00000000000001b9 0x00000000000001b9 R E       0x1000
LOAD           0x0000000000000200 0x0000000000000200 0x0000000000000200

Segment Sections...
00
01      .interp
02      .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .
gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt
03      .init .plt .plt.got .plt.sec .text .fini
04      .rodata .eh_frame_hdr .eh_frame
```

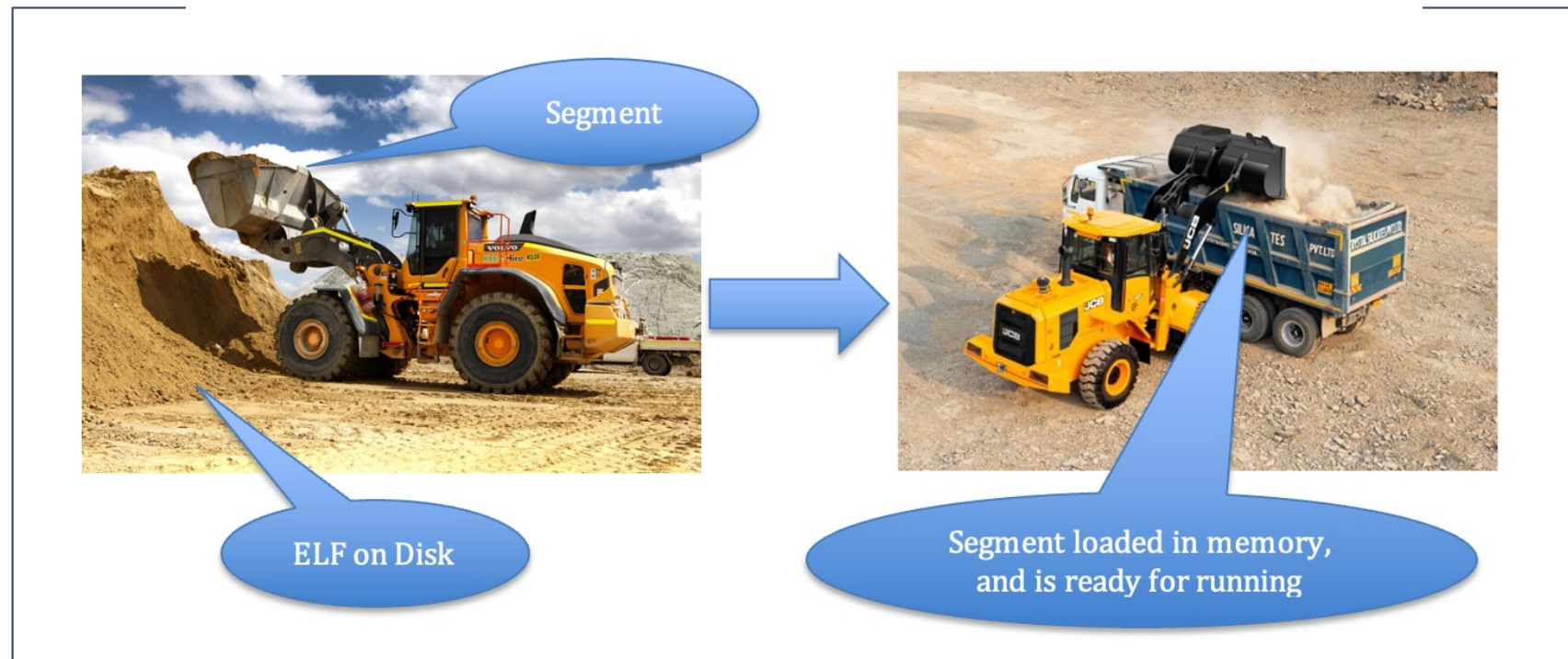
- Section “`.interp`” stores the path to the interpreter (`/lib/ld-linux.so`) that contains a **dynamic linker/loader**
 - The dynamic linking is for binding the references to shared library functions to the actual definitions (code) in the shared library
 - The dynamic loader loads shared libraries and then jumps to `_start`

Loader in OS: In Reality (2/2)

- When an executable file is run, the operating system loads it into memory and start it running as follows
 1. OS kernel reads the PHT and loads the parts specified in the segments of type “INTERP” and “LOAD”
 2. Control is transferred to the interpreter (/lib/ld-library.so)
 3. Dynamic Linker (DL) is invoked
 4. DL load the shared libraries needed by the executable and carry out the relocations
 - This information is available in “.dynamic” section
 - Connects symbolic references with their corresponding symbolic definitions (e.g., connecting the call to printf with the actual implementation of the printf in GNU C library)
 5. DL transfer the control to the address given by the symbol “_start” (and eventually to user main)

Assignment-1 (Section-A)

SimpleLoader: An ELF Loader in C from Scratch



Today's Lecture

- Loader
- **Call stack**
- Procedure calling convention

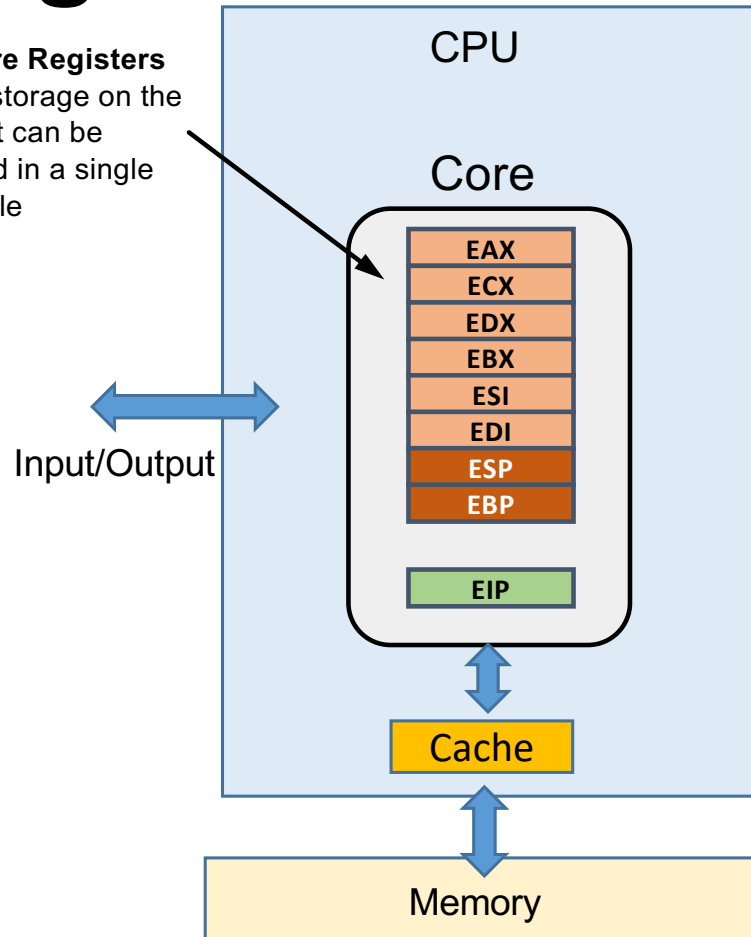
Instruction Set Architectures (ISA)

- ISA provides a precise description of features provided by the hardware and how software can invoke and access these features
 - Can be thought of a programmer's manual to run programs on a CPU
 - Interface between the hardware and the software
 - Hides the hardware complexity
- Some of the widely used ISAs
 - **x86** (or **IA32**, i386) – Developed by Intel and widely used in desktop/laptop processors
 - Throughout this course we would only refer to 32-bit x86 (or IA32)
 - x86-64 (or AMD64) – 64-bit version of x86 ISA
 - ARM – Widely used on processors for mobile devices due to their reduced instruction set, which results in small die area and low power consumption

Registers in IA32

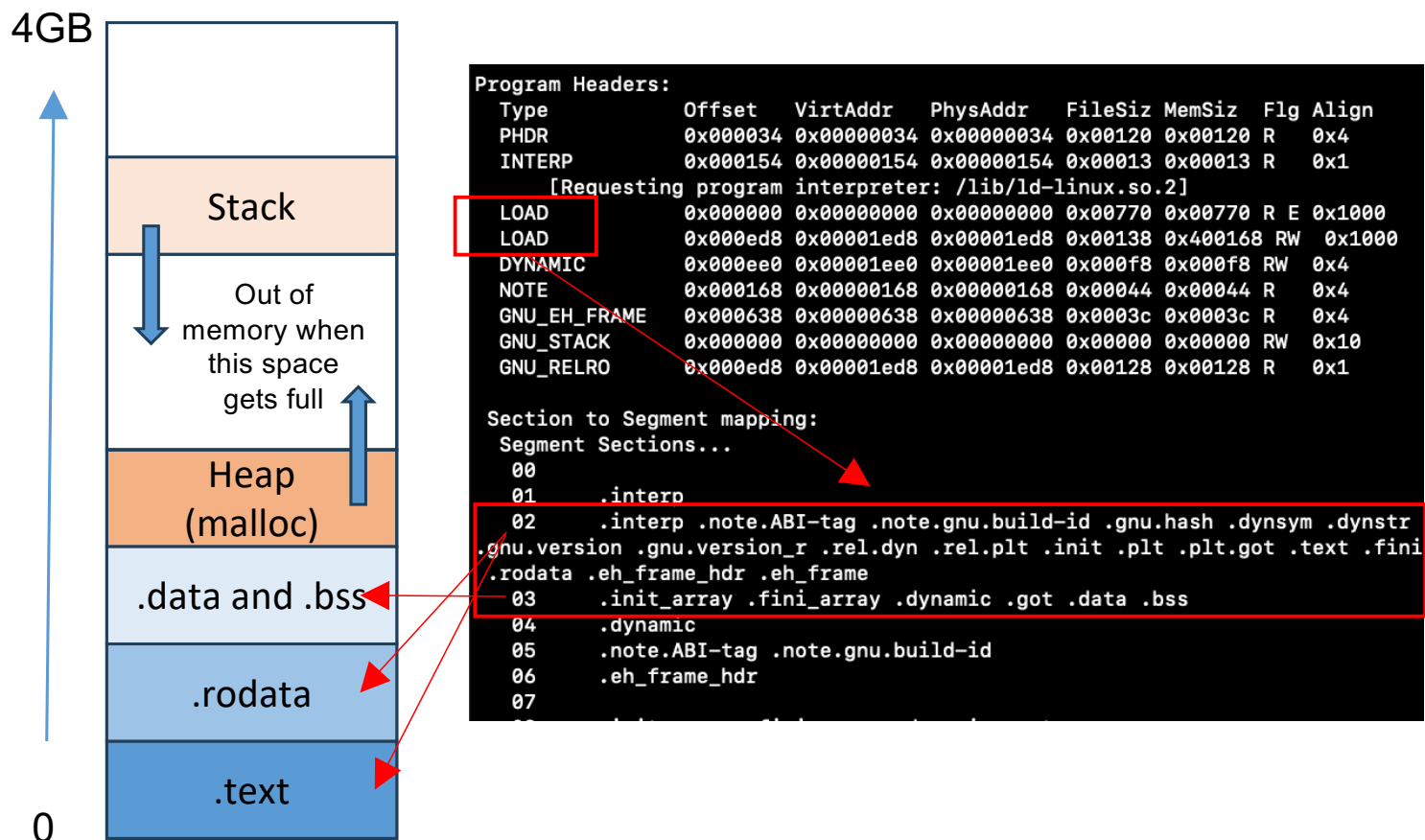
Hardware Registers

A small storage on the CPU that can be accessed in a single CPU cycle



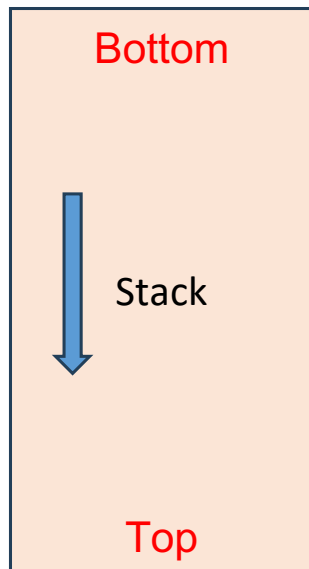
- There are total of eight data registers where six are general purpose and two are special purpose (ESP and EBP)
- EIP is a control register to hold the program counter

Memory Layout of Program in Execution



- Total 2^{32} unique memory address on a 32-bit machine
- Stack grows downward whereas heap grows upward
 - Out of memory error generated when there is no more space left. You can easily get this runtime exception while running a Java program. **Why?**
- Each of the slots shown in the figure consists of several memory pages (4K)
- Stack is the place where the intermediate state of program execution is stored
 - The cook executes instructions in the recipe one by one. He remembers the main step. After finishing the subtask, he returns back to the main step!
- Which of the memory slots are Readable? Executable? Writable?

Upside Down Call Stack in IA32



- Stack is just a region of memory allocated by the OS for a running program to save the temporary variables inside a method and to support method invocation
 - Stack grows downward
- Used to hold temporary program state
 - Method call stack (callee and caller information)
 - Local variables
 - Function arguments
 - Return address
 - Temporary space
- This **convention** of saving and restoring program state across method calls is called procedure call linkage. We will now see how it happens on IA32

Callee and Caller

L1: main () {

L2: **foo**();

L3: }

L4: **foo** () {

L5: **bar**(100, 200);

L6: }

L7: **bar**(int x1, int x2) {

L8: int b1=x1,b2=x2, b3;

L9: b3 = **baz**();

L10: }

L11: **int baz** () {

L12: int r = 10;

L13: return r;

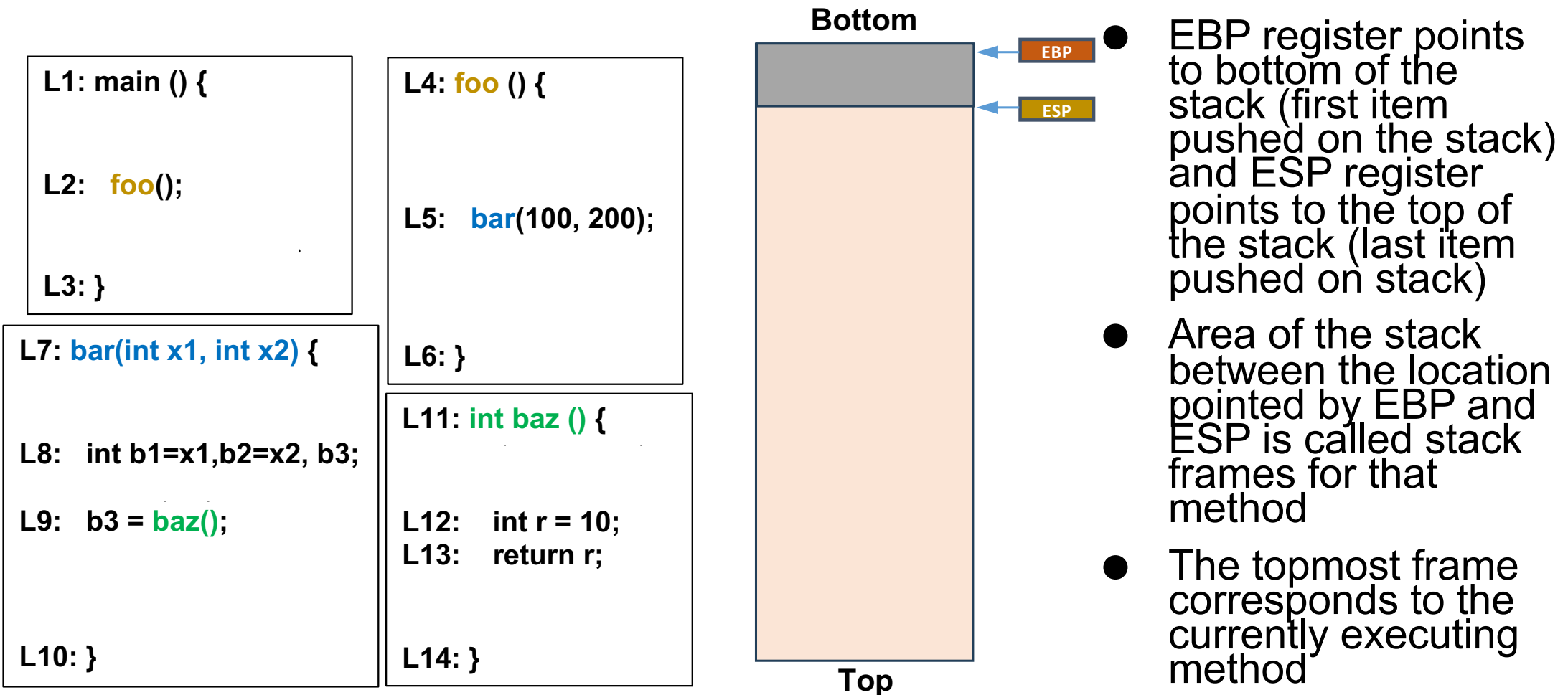
L14: }

- Call chain
 - main → foo → bar → baz
- Callee and Caller
 - main (caller) → foo (callee)
 - foo (caller) → bar (callee)
 - bar (caller) → baz (callee)

Today's Lecture

- Loader
- Call stack
- **Procedure calling convention**

Stack Frames in IA32 (1/7)



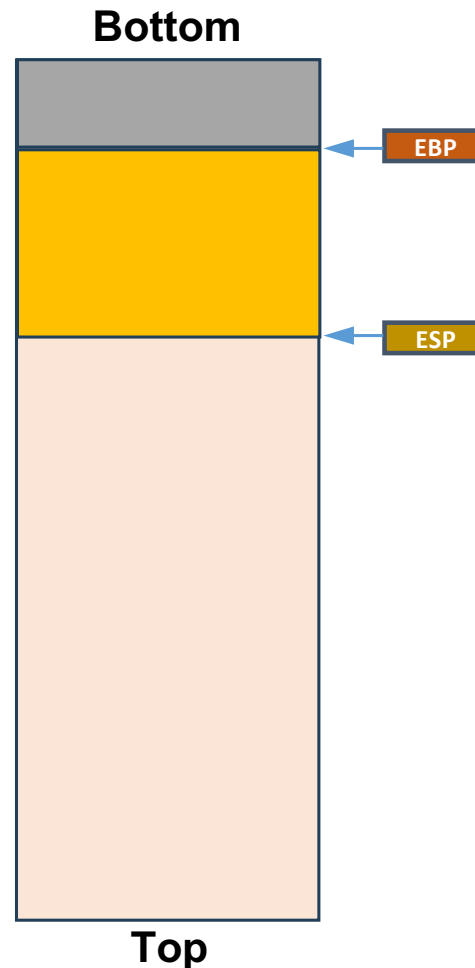
Stack Frames in IA32 (2/7)

```
L1: main () {
L2:  foo();
L3: }
```

```
L4: foo () {
L5:  bar(100, 200);
L6: }
```

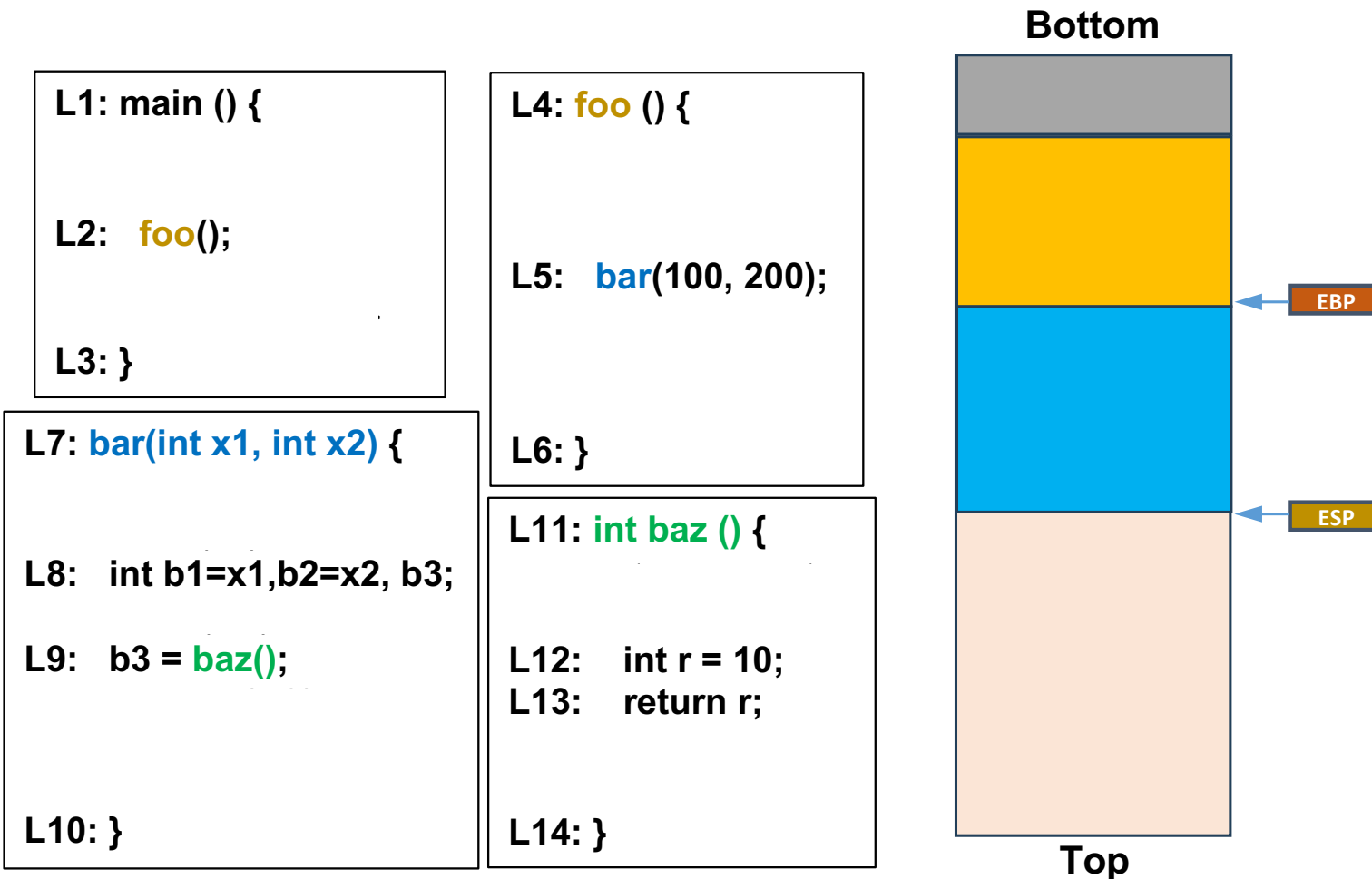
```
L7: bar(int x1, int x2) {
L8:  int b1=x1,b2=x2, b3;
L9:  b3 = baz();
L10: }
```

```
L11: int baz () {
L12:  int r = 10;
L13:  return r;
L14: }
```



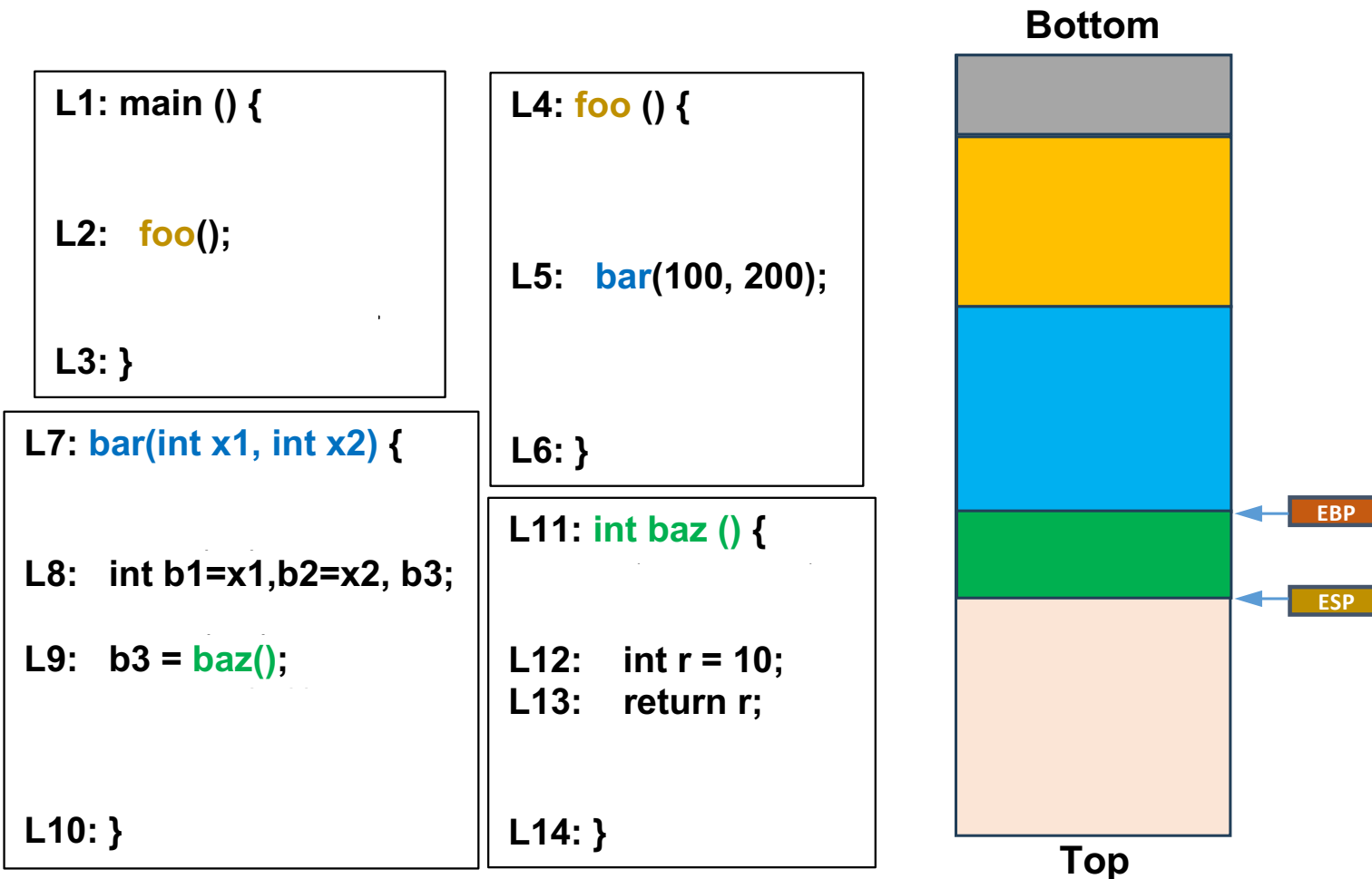
- EBP register points to bottom of the stack (first item pushed on the stack) and ESP register points to the top of the stack (last item pushed on stack)
- Area of the stack between the location pointed by EBP and ESP is called stack frames for that method
- The topmost frame corresponds to the currently executing method

Stack Frames in IA32 (3/7)



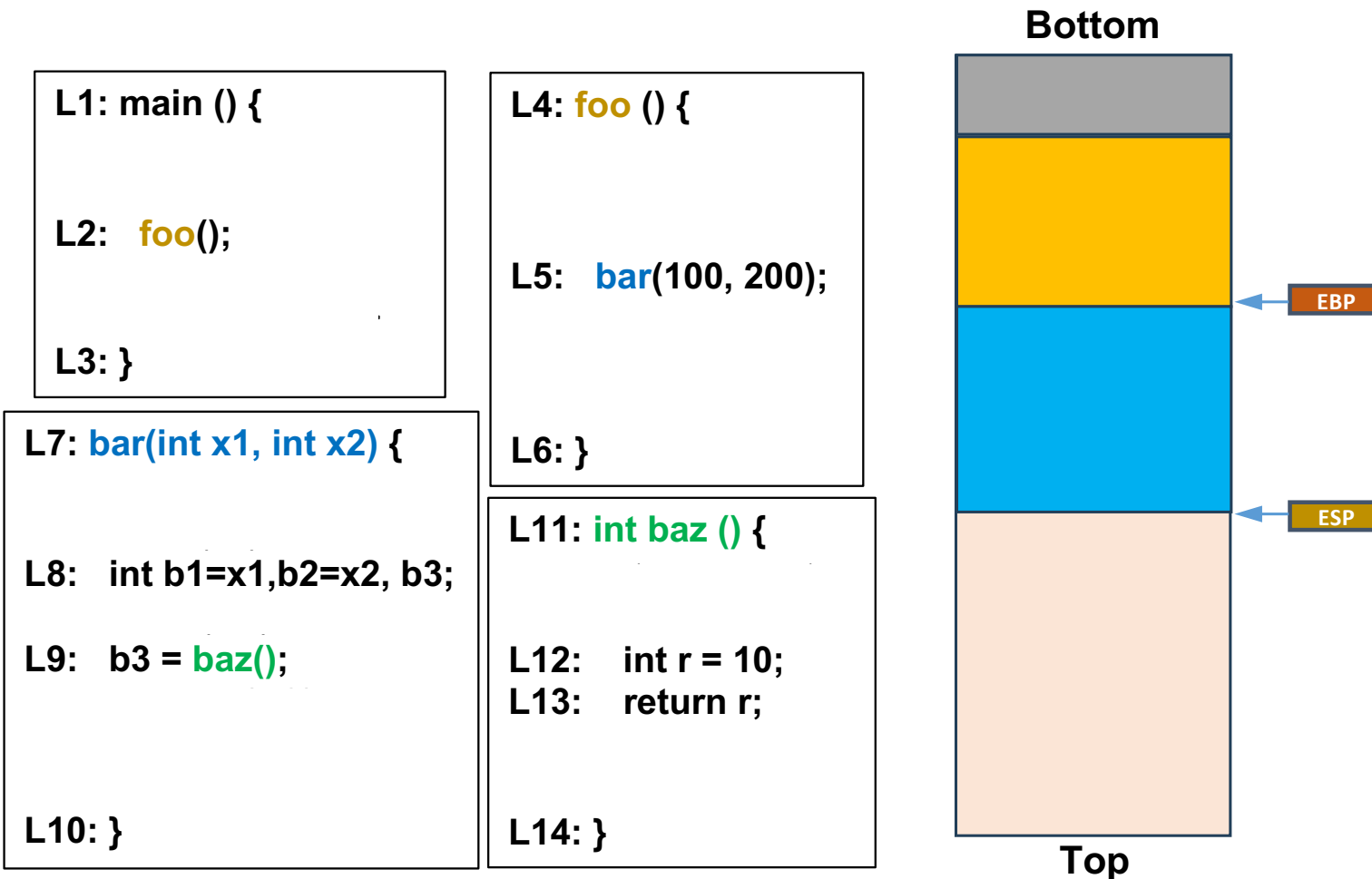
- EBP register points to bottom of the stack (first item pushed on the stack) and ESP register points to the top of the stack (last item pushed on stack)
- Area of the stack between the location pointed by EBP and ESP is called stack frames for that method
- The topmost frame corresponds to the currently executing method

Stack Frames in IA32 (4/7)



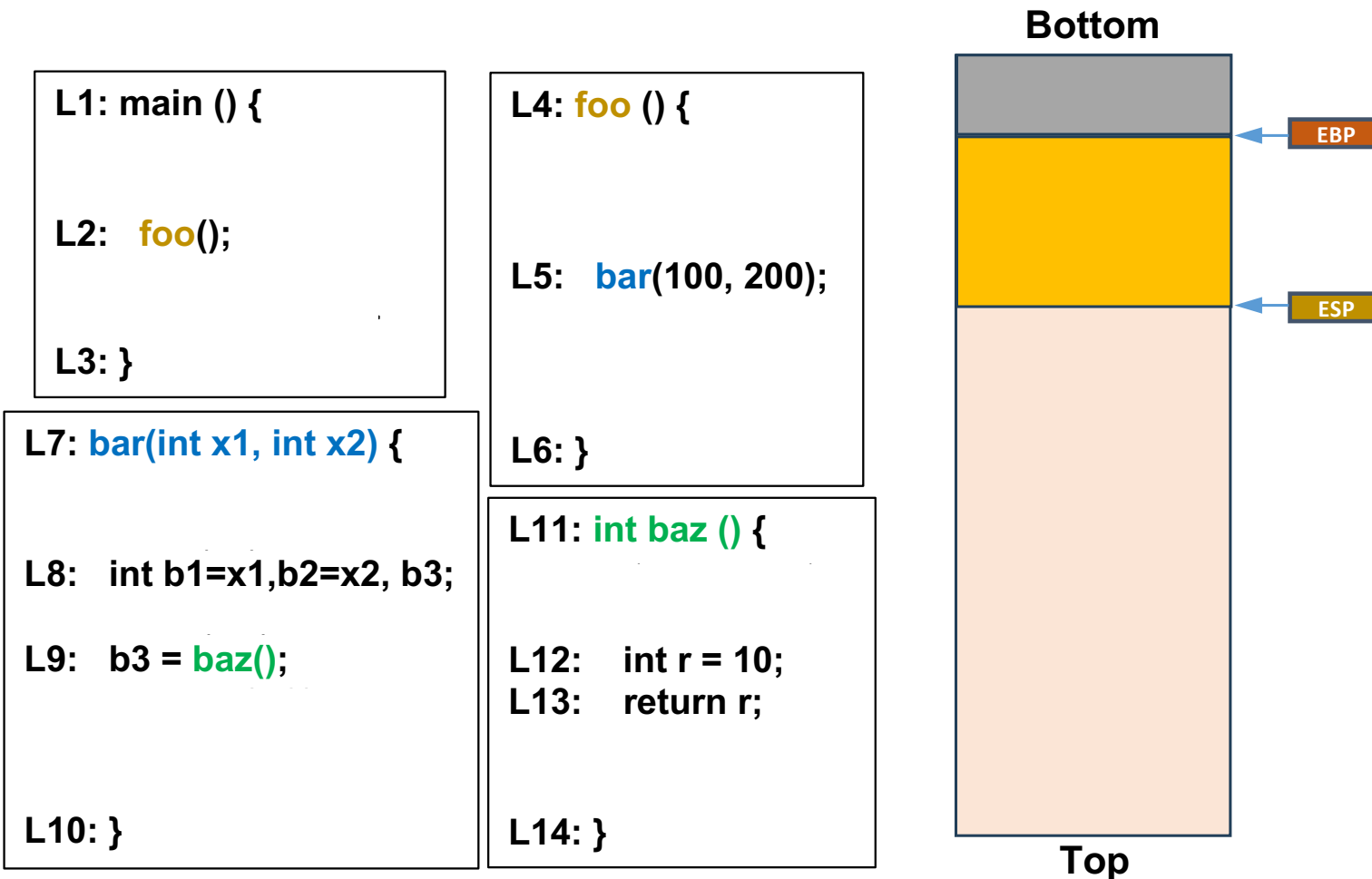
- EBP register points to bottom of the stack (first item pushed on the stack) and ESP register points to the top of the stack (last item pushed on stack)
- Area of the stack between the location pointed by EBP and ESP is called stack frames for that method
- The topmost frame corresponds to the currently executing method

Stack Frames in IA32 (5/7)



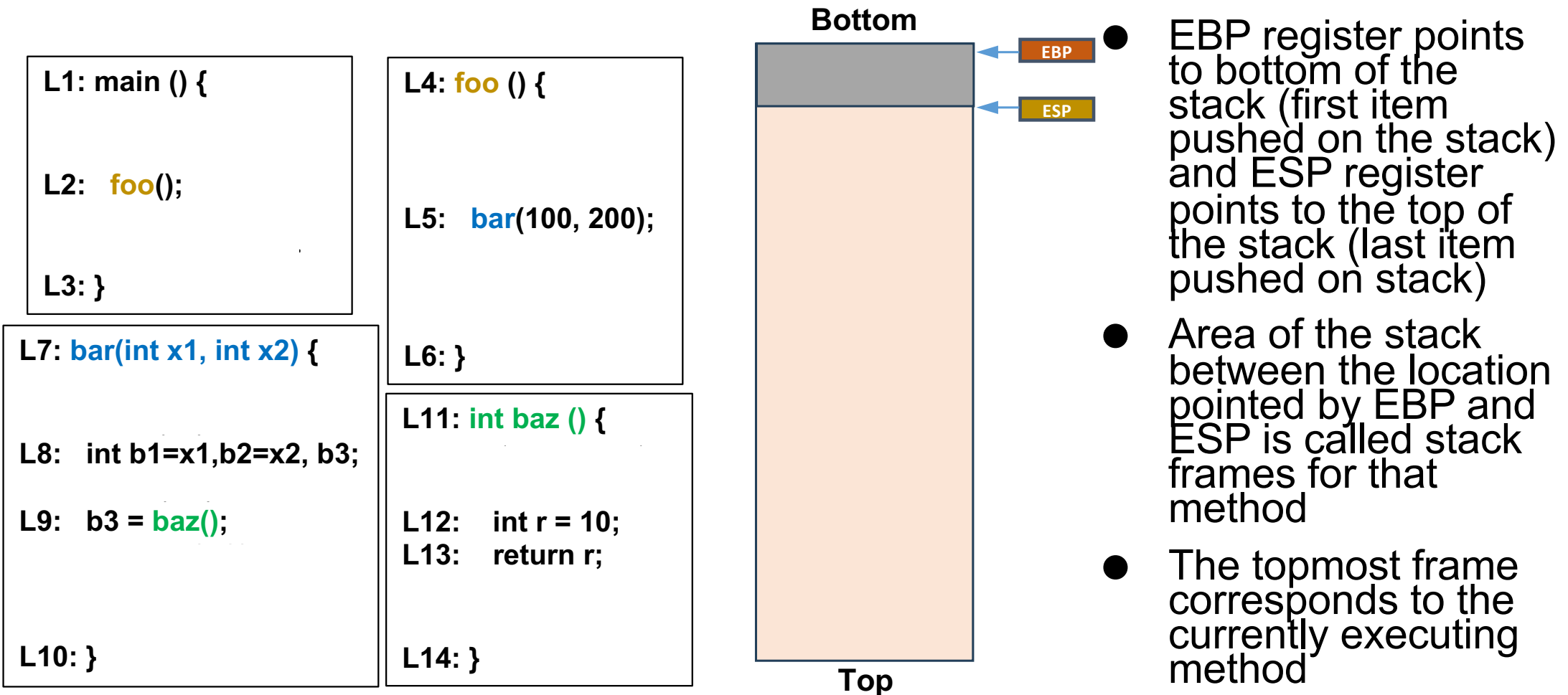
- EBP register points to bottom of the stack (first item pushed on the stack) and ESP register points to the top of the stack (last item pushed on stack)
- Area of the stack between the location pointed by EBP and ESP is called stack frames for that method
- The topmost frame corresponds to the currently executing method

Stack Frames in IA32 (6/7)

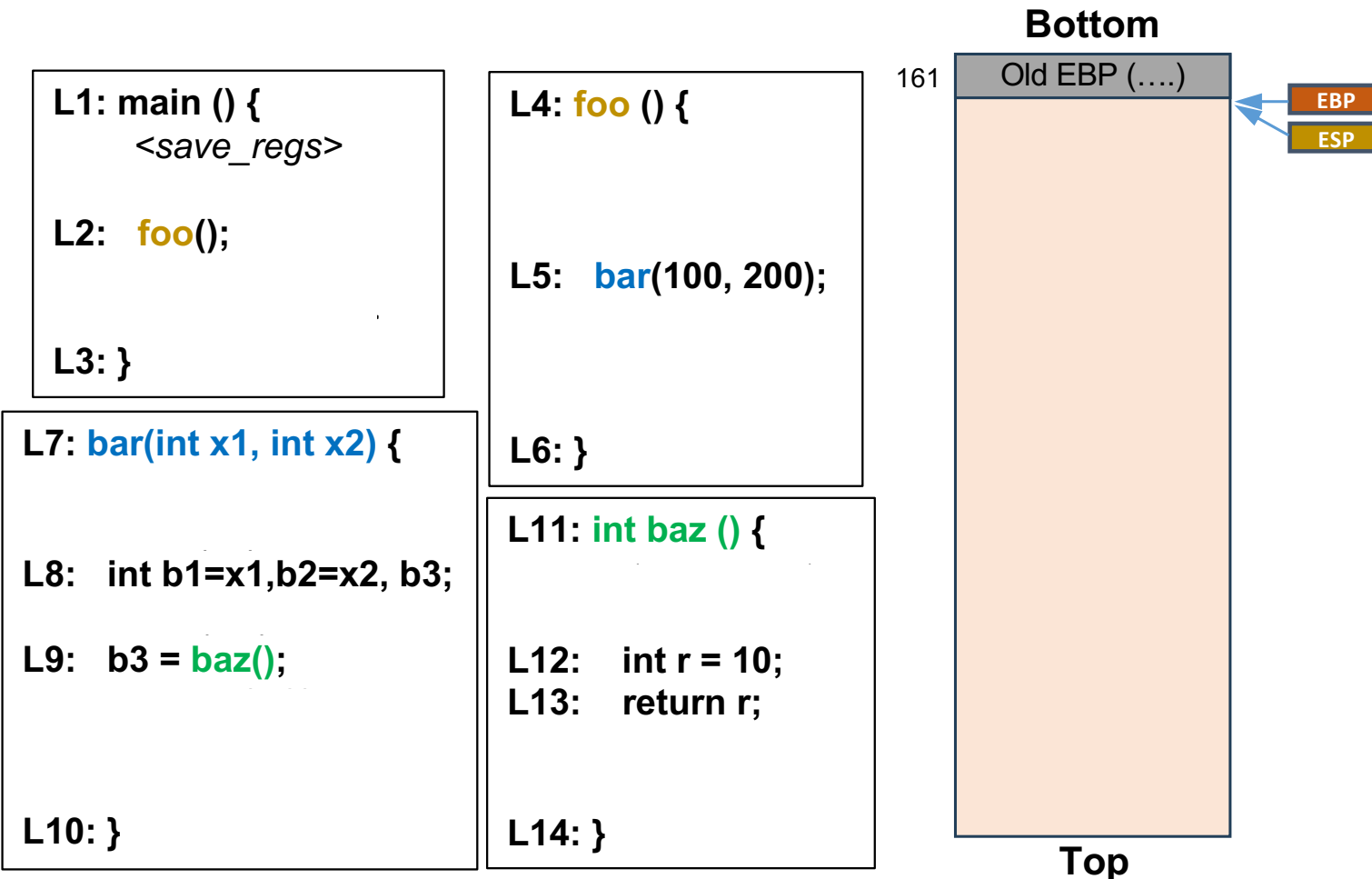


- EBP register points to bottom of the stack (first item pushed on the stack) and ESP register points to the top of the stack (last item pushed on stack)
- Area of the stack between the location pointed by EBP and ESP is called stack frames for that method
- The topmost frame corresponds to the currently executing method

Stack Frames in IA32 (7/7)



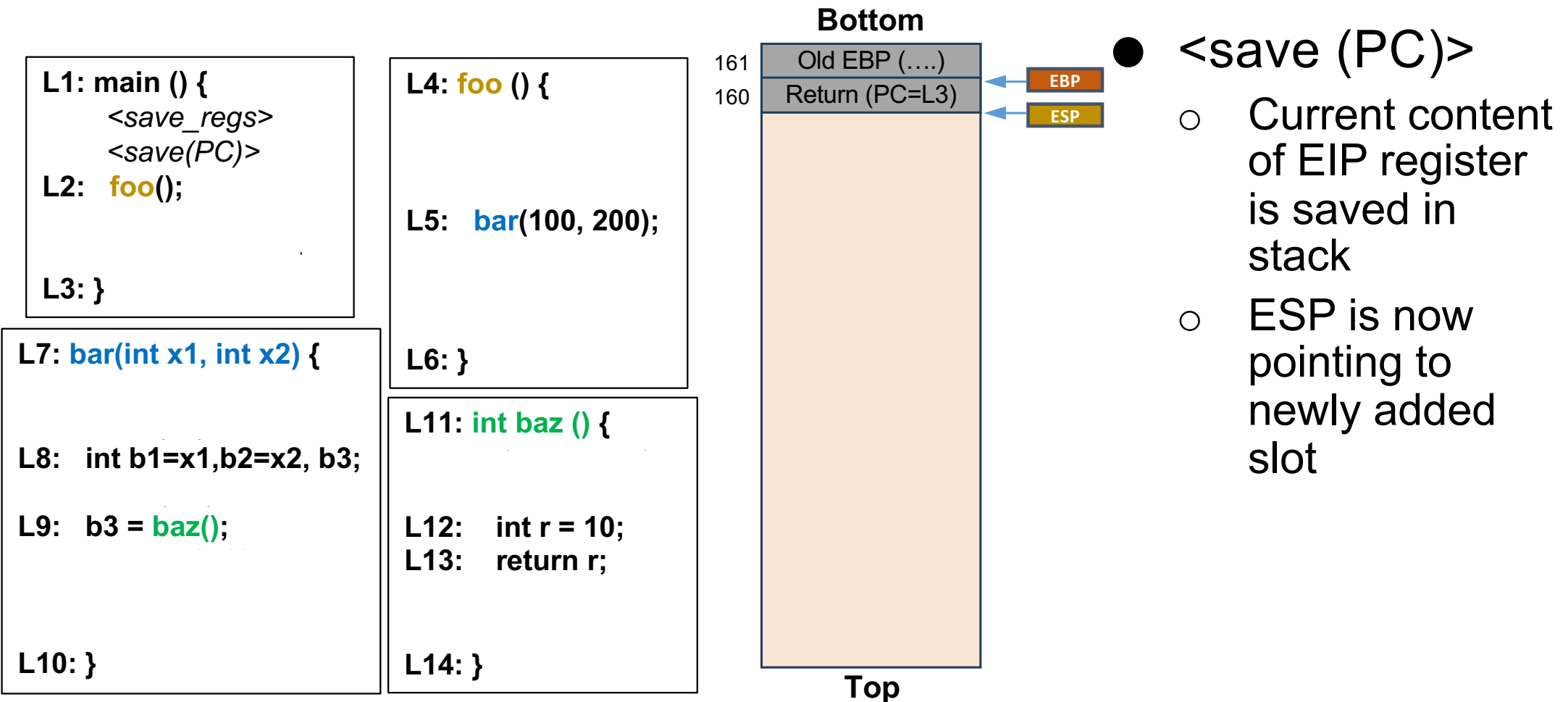
Calling Convention in X86 (1/14)



• <save regs>

- Current content of EBP register is saved in stack
 - Which method it corresponds to?
 - Carried out before every method call
 - Also known as function prologue
- EBP now points to the slot containing old value of EBP
- ESP also points to the same slot

Calling Convention in X86 (2/14)



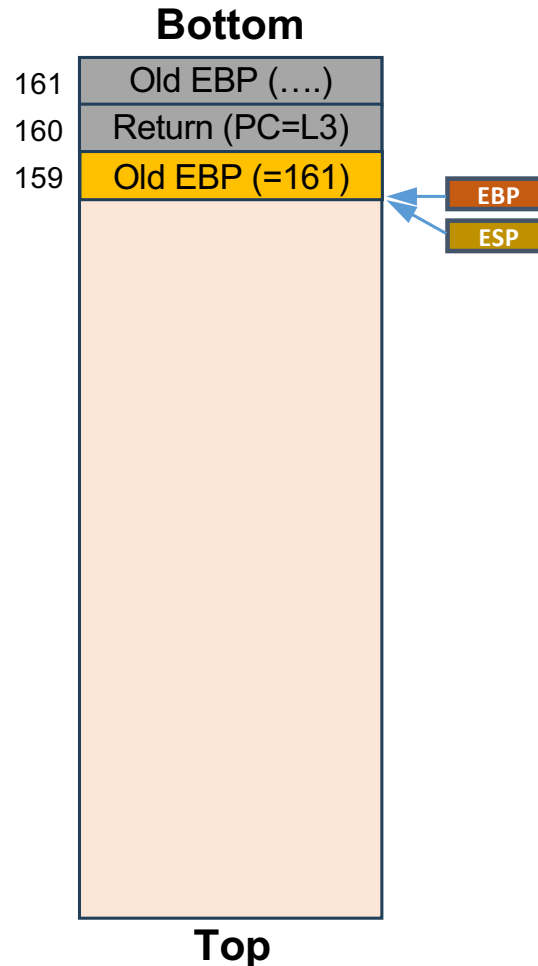
Calling Convention in X86 (3/14)

```
L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }
```

```
L4: foo () {
    <save_regs>
L5:  bar(100, 200);
L6: }
```

```
L7: bar(int x1, int x2) {
L8:  int b1=x1,b2=x2, b3;
L9:  b3 = baz();
L10: }
```

```
L11: int baz () {
L12:  int r = 10;
L13:  return r;
L14: }
```



- **<save regs>**
 - Current content of EBP register is saved in stack
 - EBP now points to the slot containing old value of EBP
 - ESP also points to the same slot

Calling Convention in X86 (4/14)

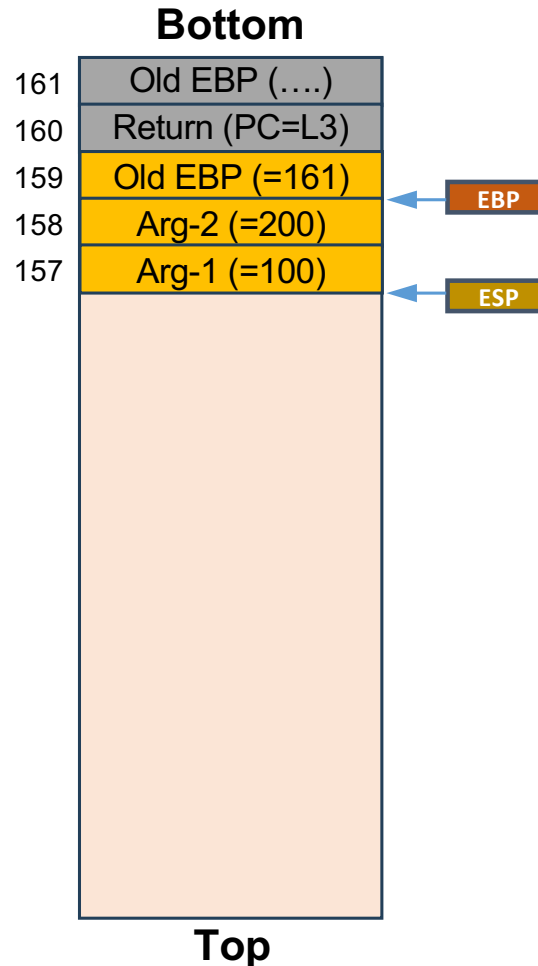
```
L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }
```

```
L4: foo () {
    <save_regs>
    <alloc(8)>
L5:  bar(100, 200);
```

```
L6: }
```

```
L7: bar(int x1, int x2) {
L8:  int b1=x1,b2=x2, b3;
L9:  b3 = baz();
L10: }
```

```
L11: int baz () {
L12:  int r = 10;
L13:  return r;
L14: }
```



- **<alloc(8)>**
 - Two slots are added on the stack in the current stack frame for two arguments to the callee method
 - Arguments are stored in the reverse ordered on the stack
 - ESP is updated

Calling Convention in X86 (5/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();

L3: }
  
```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
  
```

```

L7: bar(int x1, int x2) {

L8:  int b1=x1,b2=x2, b3;

L9:  b3 = baz();

L10: }
  
```

```

L6: }
  
```

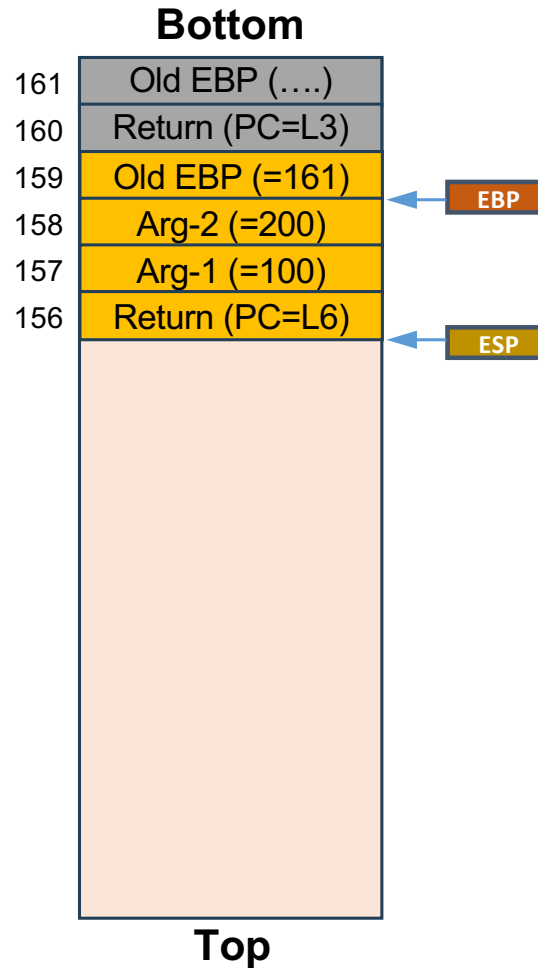
```

L11: int baz () {

L12:  int r = 10;
L13:  return r;
  
```

```

L14: }
  
```



- **<save (PC)>**
 - Current content of EIP register is saved in stack
 - ESP is now pointing to newly added slot

Calling Convention in X86 (6/14)

```
L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }
```

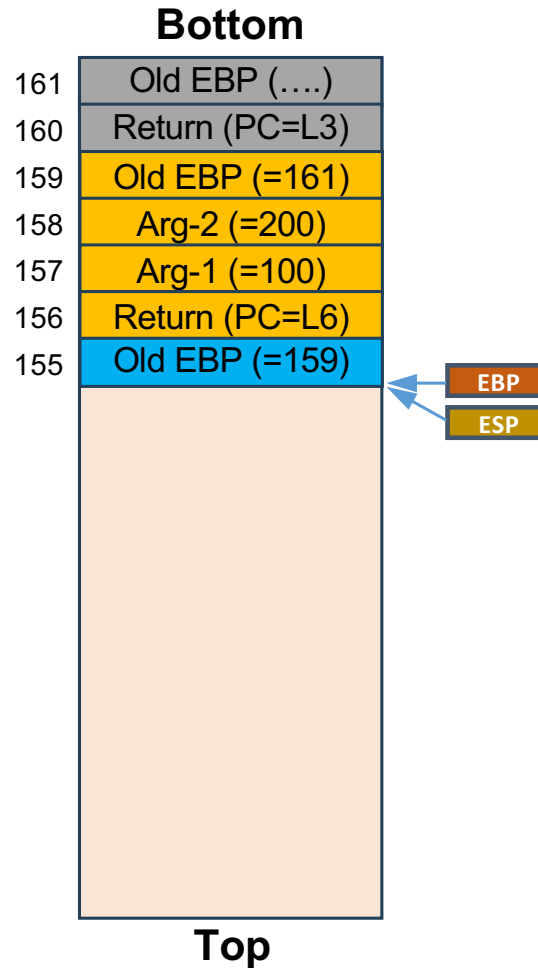
```
L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
```

```
L6: }
```

```
L7: bar(int x1, int x2) {
    <save_regs>
L8:  int b1=x1,b2=x2, b3;
L9:  b3 = baz();
L10: }
```

```
L11: int baz () {
L12:  int r = 10;
L13:  return r;
```

```
L14: }
```



- **<save regs>**
 - Current content of EBP register is saved in stack
 - EBP now points to the slot containing old value of EBP
 - ESP also points to the same slot

Calling Convention in X86 (7/14)

```
L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }
```

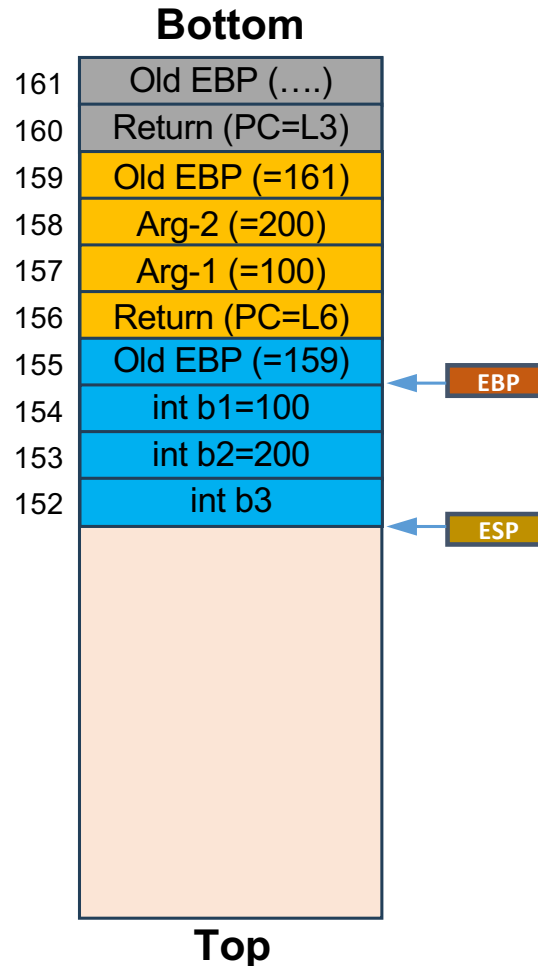
```
L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
```

```
L6: }
```

```
L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
L9:  b3 = baz();
L10: }
```

```
L11: int baz () {
L12:  int r = 10;
L13:  return r;
```

```
L14: }
```



- **<alloc(12)>**
 - Three slots are added on the stack in the current stack frame for the three local variables
 - Variables are stored in the same order on the stack as encountered
 - ESP is updated

Calling Convention in X86 (8/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }

```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);

```

```

L6: }

```

```

L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
L10: }

```

```

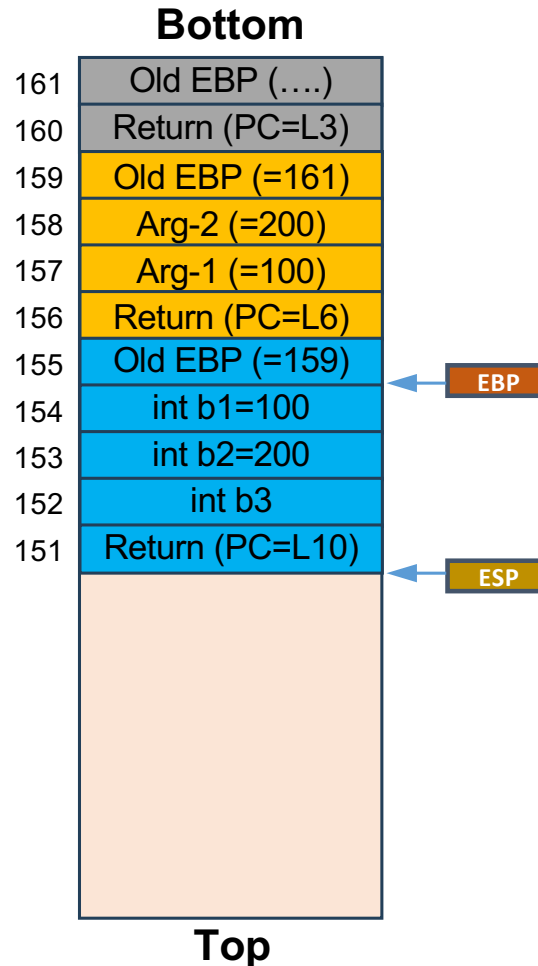
L11: int baz () {
L12:  int r = 10;
L13:  return r;

```

```

L14: }

```



- **<save (PC)>**
 - Current content of EIP register is saved in stack
 - ESP is now pointing to newly added slot

Calling Convention in X86 (9/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();

L3: }
  
```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
  
```

```

L6: }
  
```

```

L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();

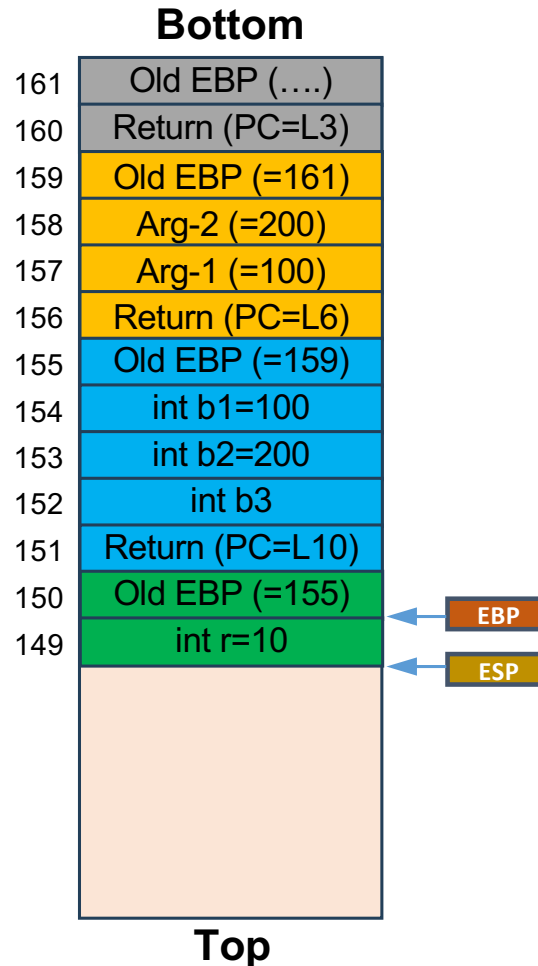
L10: }
  
```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
  
```

```

L14: }
  
```



- **<save regs>**
 - Current content of EBP register is saved in stack
 - EBP now points to the slot containing old value of EBP
 - ESP also points to the same slot
- **<alloc(4)>**
 - A slot is added on the stack in the current stack frame for the local variable

Calling Convention in X86 (10/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }

```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);

```

```

L6: }

```

```

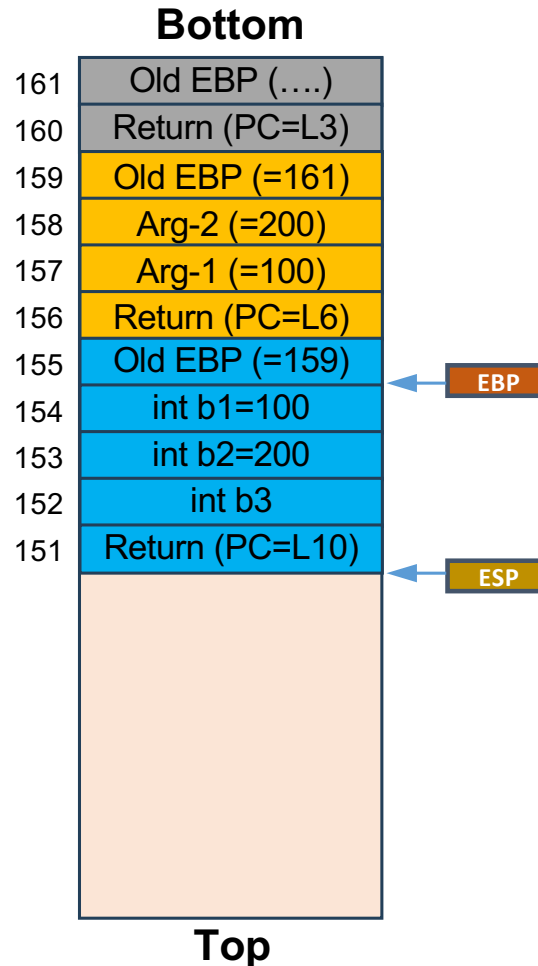
L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
L10: }

```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
    <dealloc(4)>
    <restore_regs>
L14: }

```



- Value returned is stored inside the slot 152 for “b3”
- <dealloc(4)>
 - ESP is updated to point to the slot just before the slot(s) added for storing local variable(s)
- <restore_regs>
 - Old value of EBP is popped and stored in EBP
 - ESP adjusted to point to the last slot in the current stack frame
 - Carried out before exiting any method call
 - Also known as function epilogue

Calling Convention in X86 (11/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }

```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);

```

```

L6: }

```

```

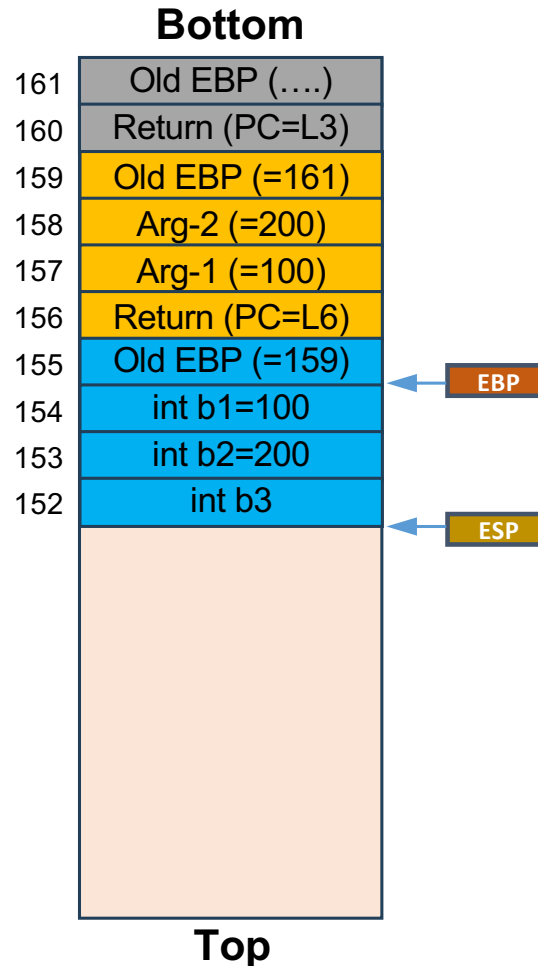
L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
    <restore(PC)>
L10: }

```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
    <dealloc(4)>
    <restore_regs>
L14: }

```



- <restore(PC)>
 - EIP register is now restored with the old value of EIP before the call went inside the callee

Calling Convention in X86 (12/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }

```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);

```

```

L6: }

```

```

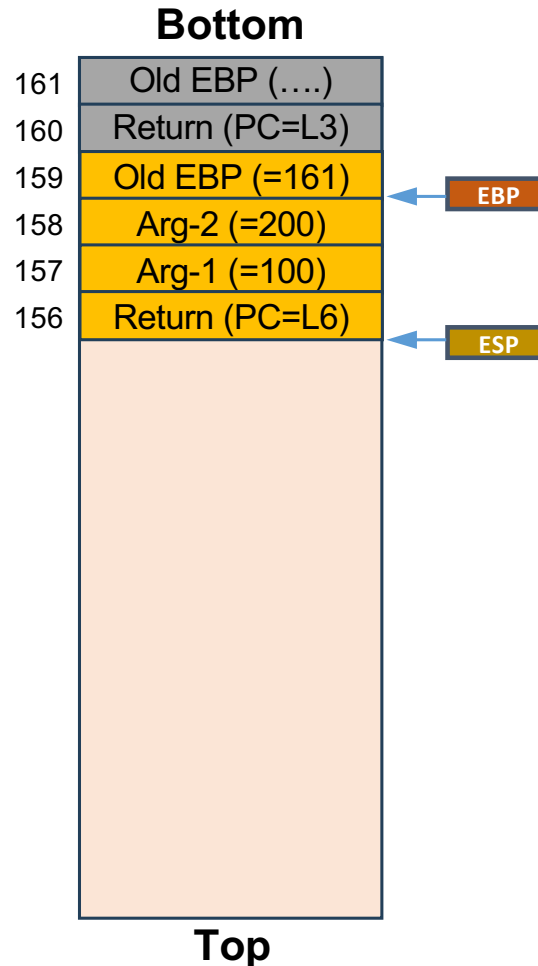
L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
    <restore(PC)>
    <dealloc(12)>
    <restore_regs>
L10: }

```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
    <dealloc(4)>
    <restore_regs>
L14: }

```



- **<dealloc(12)>**
 - ESP is updated to point to the slot just before the slot(s) added for storing local variable(s)
- **<restore_regs>**
 - Old value of EBP is popped and stored in EBP
 - ESP adjusted to point to the last slot in the current stack frame

Calling Convention in X86 (13/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
L3: }

```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
    <restore(PC)>
    <dealloc(8)>
    <restore_regs>
L6: }

```

```

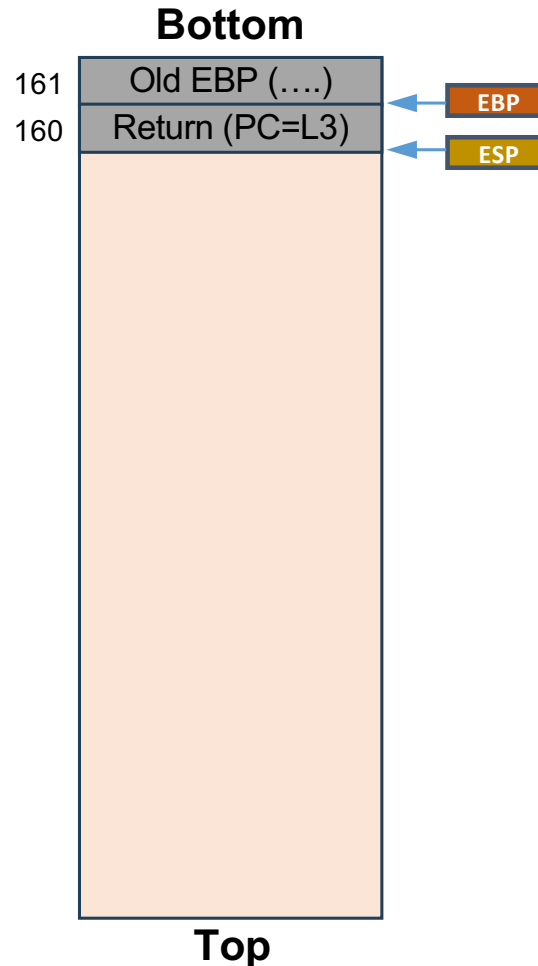
L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
    <restore(PC)>
    <dealloc(12)>
    <restore_regs>
L10: }

```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
    <dealloc(4)>
    <restore_regs>
L14: }

```



- **<restore(PC)>**
 - EIP register is now restored with the old value of EIP before the call went inside the callee
- **<dealloc(8)>**
 - ESP is updated to point to the slot just before the slot(s) added for storing local variable(s)
- **<restore_regs>**
 - Old value of EBP is popped and stored in EBP
 - ESP adjusted to point to the last slot in the current stack frame

Calling Convention in X86 (14/14)

```

L1: main () {
    <save_regs>
    <save(PC)>
L2:  foo();
    <restore(PC)>
    <restore_regs>
L3: }
  
```

```

L4: foo () {
    <save_regs>
    <alloc(8)>
    <save(PC)>
L5:  bar(100, 200);
    <restore(PC)>
    <dealloc(8)>
    <restore_regs>
L6: }
  
```

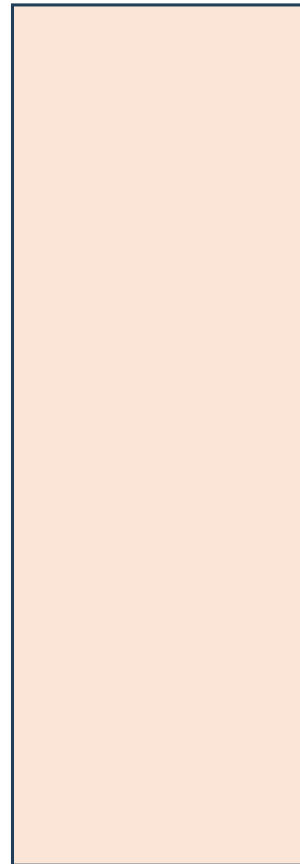
```

L7: bar(int x1, int x2) {
    <save_regs>
    <alloc(12)>
L8:  int b1=x1,b2=x2, b3;
    <save(PC)>
L9:  b3 = baz();
    <restore(PC)>
    <dealloc(12)>
    <restore_regs>
L10: }
  
```

```

L11: int baz () {
    <save_regs>
    <alloc(4)>
L12:  int r = 10;
L13:  return r;
    <dealloc(4)>
    <restore_regs>
L14: }
  
```

Bottom



Top

- As the main method terminates, call is transferred to the caller that happens to be the libc main and then finally to the `_start` method

Caller and Callee Save Registers

- Recall, there are six general purpose registers that can be used to store temporary data during a method execution
- If the caller is using these registers, it must save each of these 6 registers before transferring the call to the callee
- However, it could be possible that callee doesn't use all these registers. Hence, the saving and restoring of these 6 registers is divided across both caller and the callee
- If the caller method is using these data registers then it saves/restores only these 3 on its call stack before/after method call: **EAX, EDX, and ECX**
- If the callee method wants to use these 3 registers then it saves/restores the current value on its call stack at entry/exit: **EBX, ESI, and EDI**

x86-64 Calling Convention

- Different than IA32 (x86)
- There are 16 general purpose 64-bit registers compared to 8, 32-bit registers on x86. Hence, less dependence on stack
 - As example, the arguments to callee and temporary variables can be stored on registers
 - If more data than registers then fall back to x86 way of storing on stack
- Execution is much faster due to less dependence on stack

Next Lecture

- The process abstraction
- Quiz-1 during lecture hours (25th August)
 - Syllabus: Lectures 2-4